

Benchmark technologique

Overdrive

Aventix
Epitech Paris
13 février 2026

Sommaire

OverDrive – Benchmark technologique

1- Contexte du projet OverDrive

2- Objectifs du benchmark technologique

3- Rappel des usages cibles pour la phase de test

4- Benchmark de la brique VR

- 1- Scénario commun de test (VR)
- 2- Comparatif des solutions VR
- 3- Lecture qualitative (premium / faisabilité)

5. Backend - benchmark Node/TypeScript vs Go

- 1- Périmètre backend de référence
- 2- Scénario commun de test (backend)
- 3- Comparatif backend

6- Mobile - benchmark des technologies

- 1- Périmètre mobile (compagnon)
- 2- Scénario commun de test (mobile)
- 3- Comparatif mobile

7- Base de données - benchmark

- 1- Périmètre DB
- 2- Comparatif DB / ORM

8. Conclusion - Synthèse et choix réalistes

1- Contexte du projet OverDrive

OverDrive est un projet étudiant visant à concevoir une **plateforme immersive** autour du sport automobile, combinant **réalité virtuelle** et **visualisation de données temps réel**.

L'enjeu principal est d'explorer une alternative qualitative aux interfaces 2D classiques : **présenter les informations de course (classements, écarts, pneus, météo, événements clés)** dans un environnement VR structuré, lisible et modulable.

À ce stade, l'objectif n'est pas de livrer immédiatement un produit complet à grande échelle, mais de :

- **cadrer une architecture réaliste**, cohérente avec une montée en charge progressive,
- tester différentes technologies clés (moteur VR, backend temps réel, app mobile compagnon),
- sélectionner une stack viable et pérenne pour la suite du projet.

2- Objectifs du travail actuel

La phase actuelle vise un **benchmark technologique ciblé**, principalement sur la brique VR, en prenant en compte :

- **Contraintes équipe** : 5 développeurs, 1 à 1,5 an, capacité raisonnable d'apprentissage.
- **Contraintes matérielles** : Meta Quest 3/3S en cible, machines PC hétérogènes.
- **Exigences de qualité minimales** : application fluide, faible latence, lisibilité premium, attention à l'accessibilité.

Concrètement, ce benchmark doit permettre de :

- Identifier 2 à 3 solutions VR réalistes (Unity, Unreal, WebXR).
- Mettre en place des **prototypes comparables**, représentatifs d'OverDrive.
- Mesurer, pour chaque solution : performance, facilité de développement, intégration backend, et capacité à évoluer.

L'objectif n'est pas de couvrir tous les use cases finaux, mais de **valider un socle technique** capable de supporter une évolution progressive.

3. Rappel minimal des usages cibles

Dans le cadre du benchmark, les usages sont volontairement limités et orientés “validation technique” :

- Affichage en VR de **données essentielles** (classement, écarts, événements).
- Distinction simple entre :
 - un **mode “simplifié”** (profil non expert),
 - un **mode “détaillé”** (profil expert),
via la densité et la hiérarchisation de l’information.
- **Personnalisation légère** des panneaux (position / taille) + sauvegarde via le backend.

Ce périmètre suffit à comparer efficacement des moteurs VR, sans surcharger le prototype de storytelling ou de scénarios complexes

4- Brique VR – périmètre et benchmark

1- Scénario commun de test (VR)

Toutes les solutions sont évaluées sur un scénario strictement identique :

- **Casque** : Meta Quest 3 / 3S
- **Backend** : API REST + WebSocket (backend de test ou mock)
- Scène VR :
 - une salle OverDrive (environnement sobre),
 - un **mini circuit 3D** (low poly) + **petites voitures** mises à jour quasi temps réel,
 - 2 à 4 panneaux data (classement, écarts, événements),
 - 1 panneau vidéo (replay / live selon disponibilité).

2- Comparatif des solutions VR

Solution	Ce qu’on teste concrètement	Points forts attendus	Limites / risques probables
Unity (natif Quest)	Scène Quest + circuit 3D + voitures + UI 3D + WebSocket + vidéo	Excellent compromis perf/ simplicité sur Quest, écosystème immense, itérations rapides, très “VR product-ready”	Demande une discipline d’optimisation (3D + UI + vidéo), pièges réseau (TLS, WS Android) à anticiper

Solution	Ce qu'on teste concrètement	Points forts attendus	Limites / risques probables
Unreal Engine	Scène équivalente, mêmes flux data + vidéo (prototype moins dense)	Très bon rendu possible, outils d'optimisation puissants	Courbe d'apprentissage lourde, souvent surdimensionné pour une app data-first, risque de timebox explosée
WebXR (browser Quest)	Expérience WebXR : circuit "light" + panneaux 3D + vidéo + WebSocket	Déploiement ultra rapide (URL), stack web homogène, parfait pour démo	Performance/UX dépendantes du navigateur, stabilité variable, vidéo + 3D + UI peut vite limiter

3- Lecture qualitative (premium / faisabilité)

- **Unity** est le candidat le plus cohérent si l'objectif est une **VR principale premium** sur Quest, avec un bon équilibre entre rendu, performance et vitesse d'itération.
- **Unreal** est pertinent si l'objectif prioritaire devient le rendu "cinématique", mais présente un risque fort en contexte étudiant (temps + complexité).
- **WebXR** est très intéressant pour une **version démo / alternative web**, mais plus risqué comme plateforme VR principale.

5- Backend – benchmark Node/TypeScript vs Go

1- Périmètre backend de référence

Le backend OverDrive expose :

- une **API REST** : comptes, presets/layouts, catalogue d'événements et métadonnées,
- un **WebSocket** : diffusion temps réel (positions, classement, événements simples).

Le backend reste agnostique des clients : VR, mobile et web consomment le **même contrat**.

2- Scénario commun de test (backend)

API REST

- Auth simplifiée : login/register
- /events, /events/:id/sessions
- /presets, /presets/:id

WebSocket

- Flux session (state, drapeaux, classement) : **realtime:session:<id>:state**
- Flux positions voitures : **realtime:session:<id>:driver:<driverId>** ou flux agrégé

Charge simulée

- 5–20 updates/s pour la session
- 1–5 updates/s par pilote suivi
- quelques clients simultanés (VR + mobile + web)

3- Comparatif backend

Solution	Ce qu'on teste	Points forts attendus	Limites / risques probables
Go	REST + WebSocket + PostgreSQL + Redis	Très bon temps réel, faible conso CPU, simplicité du modèle "ingest → broadcast", excellent pour flux fréquents	Moins de synergie "monolanguage" si front en TS, plus de boilerplate selon frameworks
Node/TypeScript	REST + WS (ws/Socket.IO) + PostgreSQL + Redis	Écosystème massif, monolanguage avec RN/web, DX fort (OpenAPI, validation, auth)	Perf brute moindre à charge égale sur WS intensif, nécessite discipline async
Mix (optionnel)	Go pour temps réel + Node pour services	Combine perf Go + confort Node	Complexité multi-services, probablement inutile en projet étudiant

Lecture recommandée

- Si l'ambition prioritaire est le **temps réel "propre"** et robuste : **Go est très cohérent.**
- Si le critère principal est l'alignement full TS (web/mobile) : **Node/TS est très viable** dans un cadre étudiant (charge limitée).

6- Mobile – benchmark des technologies

1- Périmètre mobile (compagnon)

Le mobile n'est pas la brique "showcase" principale, mais une composante premium de l'écosystème :

- suivi parallèle à la VR,
- notifications,
- mode simplifié / mode détaillé,
- accès aux temps forts.

2- Scénario commun de test (mobile)

- iOS + Android
- REST + WebSocket (mêmes endpoints que VR)
- 1 écran live : classement simplifié, écarts, événements clés
- bascule simplifié - détaillé
- exigences accessibilité : contrastes, tailles de texte, densité d'info

3- Comparatif mobile

Solution	Ce qu'on teste	Points forts attendus	Limites / risques probables
Flutter	App iOS/Android + WebSocket + thèmes + accessibilité	Très bon cross-platform, rendu performant, contrôle UI fin, rapide si base existante	Moins aligné si front web en TS, attention au rafraîchissement (batterie/perf)
React Native	App équivalente (REST/WS + UI live)	Monolanguage JS/TS avec backend Node et web, énorme écosystème	Bridging natif parfois pénible, perf à surveiller, coût d'apprentissage si équipe déjà Flutter
Swift/Kotlin natif	Écran live minimal par OS	Perfs excellentes, contrôle OS total	Temps doublé, peu réaliste sur projet multi-briques en 1-1,5 an

Recommandation (cohérente projet étudiant + premium)

- **Flutter** en candidat principal (time-to-market + qualité UI + base existante).
- RN et natif en alternatives théoriques à garder en tête, sans exploration profonde au départ.

7- Base de données – benchmark

1- Périmètre DB

- Persistant : users, auth sessions, providers, events/sessions, drivers/teams, presets/layouts
- Temps réel : Redis (état live), (option) base time-series pour historique télémétrie

2- Comparatif DB / ORM

Solution	Ce qu'on teste	Points forts attendus	Limites / risques probables
PostgreSQL + Prisma	schéma + migrations + CRUD presets/users	Typage fort, migrations propres, DX excellent en TS	Pas adapté à un backend Go
PostgreSQL + ORM Go (GORM/sqlc)	même schéma + requêtes catalogue/presets	Aligné Go, perf solides, contrôle fin	DX moins “batteries included” que Prisma
PostgreSQL + SQL “light”	requêtes manuelles	Contrôle total, pédagogie SQL, perf prévisible	Plus verbeux, risque d’erreurs, maintenance plus lourde

Recommandation simple (selon backend)

- Backend **Go** → PostgreSQL + **GORM ou sqlc** + Redis (+ option TimescaleDB plus tard).
- Backend **Node/TS** → PostgreSQL + **Prisma** + Redis (+ option time-series).

Conclusion – Synthèse et choix réalistes

Le benchmark vise à sélectionner une stack **réaliste**, maîtrisable par l'équipe, et capable de soutenir un rendu **premium** sans surinvestir dans la complexité.

Orientation recommandée (cohérente avec les contraintes et la qualité cible)

- VR principale : Unity (Quest native)
- **Backend temps réel : Go ou Node/TS** (décision guidée par l'organisation équipe, pas par le client VR)
- Mobile compagnon : Flutter
- **Data : PostgreSQL + Redis** (+ time-series optionnelle)

Cette sélection permet de prototyper vite, d'assurer une bonne qualité de rendu sur Quest, et de conserver une architecture évolutive vers des fonctionnalités plus ambitieuses (télémétrie avancée, multi-sessions, replays, etc.).